

Graph Neural Networks: Model II

Jiaxuan You

Assistant Professor at UIUC CDS



CS512: Data Mining Principles, 2025 Fall

<https://ulab-uiuc.github.io/CS512/>

Announcements

- **Reminder:** Writing task 1 due **Sept 17 11:59pm**
 - [Template](#) has been released on Canvas
- **Reminder:** Writing task 2 released, due **Sept 24 11:59pm**
 - [Template](#) has been released on Canvas
 - Focuses on **cross-paper inconsistency. Why this task?**
 - Each paper is inevitably subjective, understanding inconsistency make our understanding more objective
 - Inconsistency leads to novel research ideas!

Inconsistency leads to novel research ideas

- **Idea-level inconsistency (2.5%)**
 - Example:
 - Simplifying Graph Convolutional Networks
 - “We don’t need complicated GCN, simple feature propagation is sufficient”
 - Simple and Deep Graph Convolutional Networks
 - “No, we still need deep GCN”
- **Experiment-level inconsistency**

Inconsistency leads to novel research ideas

- Experiment-level inconsistency (5%)
 - Example:

Table 1: Reproduction of Heterogeneous GNNs with simple GCN and GAT as baselines—all reproduction experiments use official codes and the same dataset, settings, hyperparameters as the original paper. The line with star (*) are results reported in the paper, and the lines without star are our reproduction. “-” means the results are not reported in the original paper. We mark the reproduction terms with >1 point gap compared to the reported results by ↑ and ↓. We also keep the standard variance terms above 1.

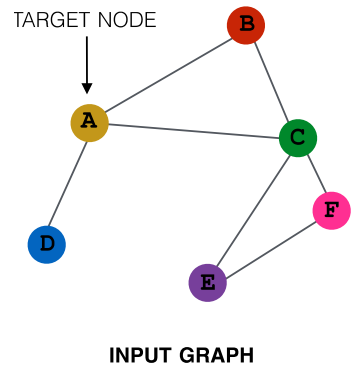
	HAN [36]		GTN [43]			RSHN [45]			HetGNN [44]				MAGNN [12]	
Dataset	ACM		DBLP	ACM	IMDB	AIFB	MUTAG	BGS	MC (10%)		MC (30%)		DBLP	
Metric	Macro-F1	Micro-F1	Macro-F1	Macro-F1	Macro-F1	Accuracy	Accuracy	Accuracy	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
model*	91.89	91.85	94.18	92.68	60.92	97.22	82.35	93.10	97.8	97.9	98.1	98.2	93.13	93.61
GCN*	89.31	89.45	87.30	91.60	56.89	-	-	-	-	-	-	-	88.00	88.51
GAT*	90.55	90.55	93.71	92.33	58.14	91.67	72.06	66.32	96.2	96.3	96.5	96.5	91.05	91.61
model	90.94	90.96	92.95↓	92.28	57.53±2.22↓	97.22	82.35	93.10	97.06	97.11	97.34	97.37	92.81	93.36
GCN	92.25↑	92.29↑	91.48↑	92.28	59.11±1.73↑	97.22	79.41	96.55	91.88	92.04	95.37	95.57	88.31	89.37
GAT	92.08↑	92.15↑	94.18	92.49	58.86±1.73	100↑	80.88↑	100↑	98.25↑	98.30↑	98.42↑	98.50↑	94.40↑	94.78↑

Reported
numbers

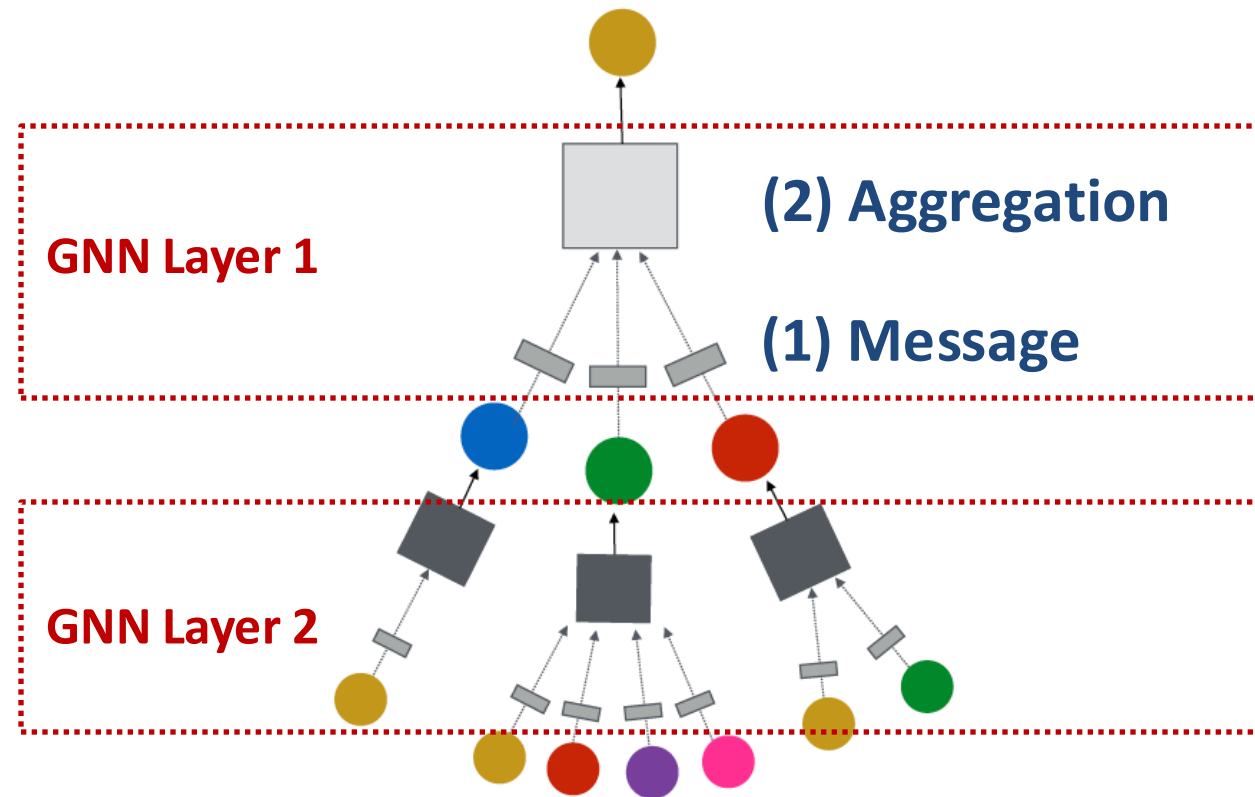
Reproduced
numbers

Recap: A General GNN Framework

(5) Learning objective



(3) Layer connectivity



(4) Graph augmentation

Recap: A Single GNN Layer

- **Putting things together:**

- **(1) Message:** each node computes a message

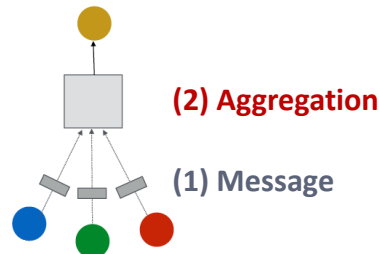
$$\mathbf{m}_u^{(l)} = \text{MSG}^{(l)} \left(\mathbf{h}_u^{(l-1)} \right), u \in \{N(v) \cup v\}$$

- **(2) Aggregation:** aggregate messages from neighbors

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\}, \mathbf{m}_v^{(l)} \right)$$

- **Nonlinearity (activation):** Adds expressiveness

- Often written as $\sigma(\cdot)$: $\text{ReLU}(\cdot)$, $\text{Sigmoid}(\cdot)$, ...
- Can be added to **message or aggregation**



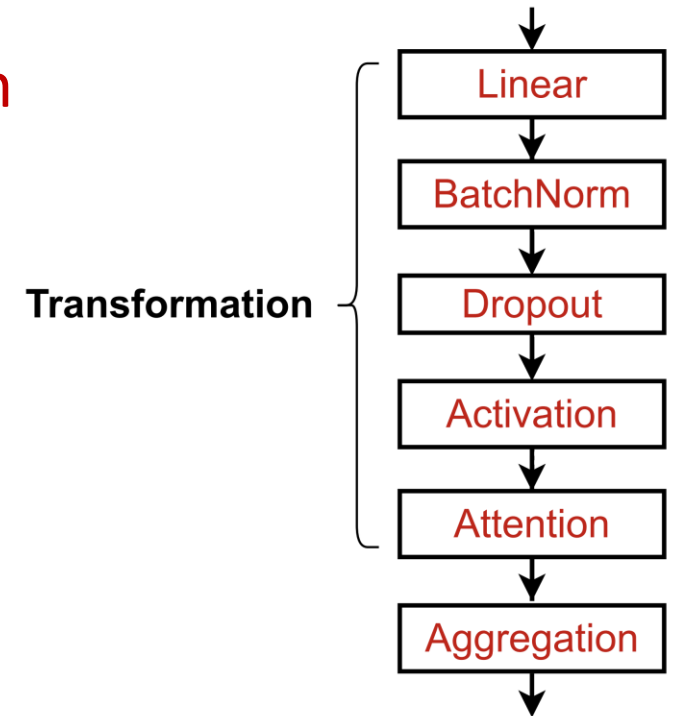
Graph Neural Networks: Model II

GNN Layers in Practice

GNN Layer in Practice

- In practice, these classic GNN layers are a great starting point
 - We can often get better performance by **considerin** a general GNN layer design
 - Concretely, we can **include modern deep learning modules** that proved to be useful in many domains

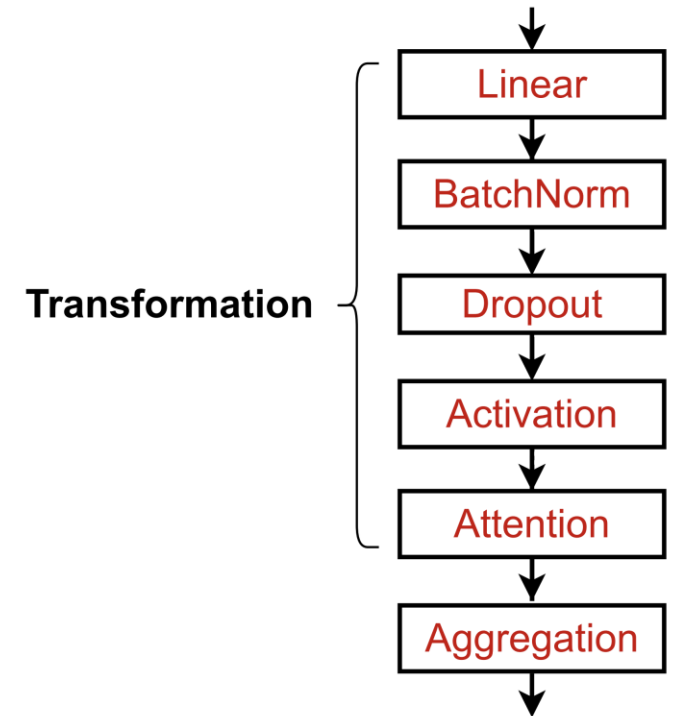
A suggested GNN Layer



GNN Layer in Practice

- Many modern deep learning modules can be incorporated into a GNN layer
 - **Batch Normalization:**
 - Stabilize neural network training
 - **Dropout:**
 - Prevent overfitting
 - **Attention/Gating:**
 - Control the importance of a message
 - **More:**
 - Any other useful deep learning modules

A suggested GNN Layer



Batch Normalization

- **Goal:** Stabilize neural networks training
- **Idea:** Given a batch of inputs (node embeddings)
 - Re-center the node embeddings into zero mean
 - Re-scale the variance into unit variance

Input: $\mathbf{X} \in \mathbb{R}^{N \times D}$
 N node embeddings

Trainable Parameters:
 $\gamma, \beta \in \mathbb{R}^D$

Output: $\mathbf{Y} \in \mathbb{R}^{N \times D}$
Normalized node embeddings

Step 1:
Compute the mean and variance over N embeddings

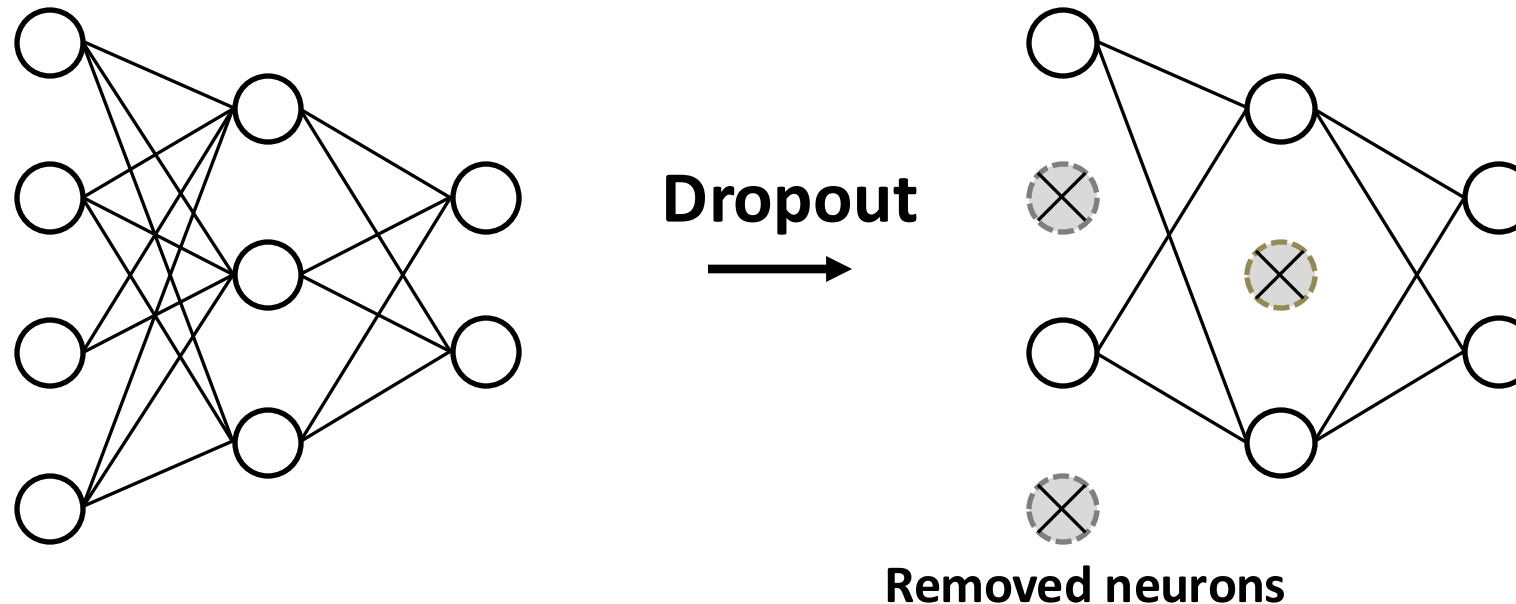
$$\mu_j = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_{i,j}$$
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_{i,j} - \mu_j)^2$$

Step 2:
Normalize the feature using computed mean and variance

$$\hat{\mathbf{x}}_{i,j} = \frac{\mathbf{x}_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$
$$\mathbf{y}_{i,j} = \gamma_j \hat{\mathbf{x}}_{i,j} + \beta_j$$

Dropout

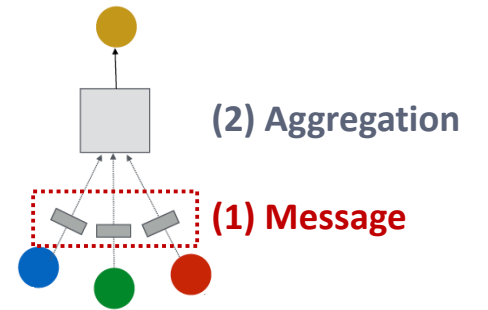
- **Goal:** Regularize a neural net to prevent overfitting.
- **Idea:**
 - **During training:** with some probability p , randomly set neurons to zero (turn off)
 - **During testing:** Use all the neurons for computation



Dropout for GNNs

- In GNN, Dropout is applied to **the linear layer in the message function**

- A simple message function with linear layer: $\mathbf{m}_u^{(l)} =$



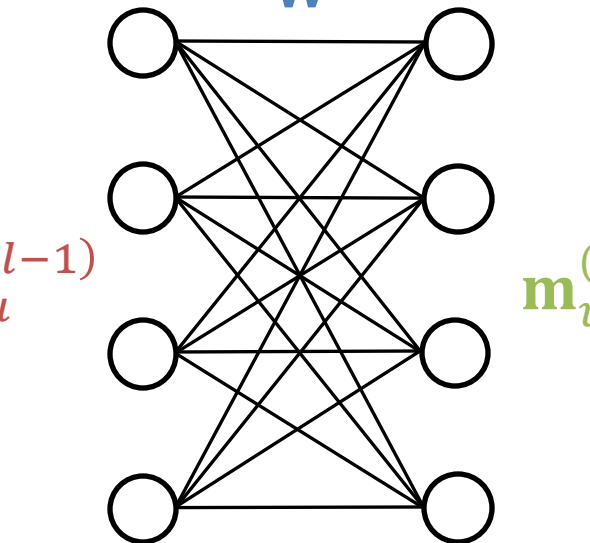
$\mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}$

$\mathbf{W}^{(l)}$

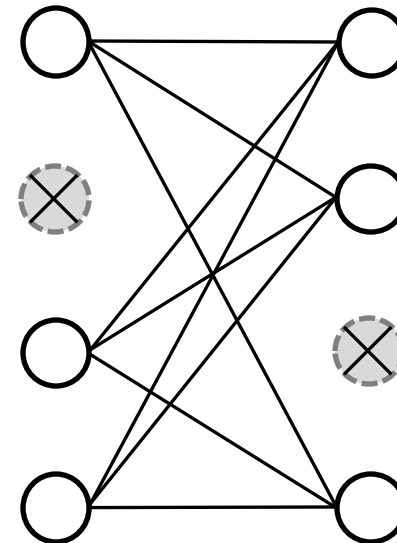
$\mathbf{h}_u^{(l-1)}$

$\mathbf{m}_u^{(l)}$

Dropout



Visualization of a linear layer



Activation (Non-linearity)

Apply activation to i -th dimension of embedding $\mathbf{x}$

- **Rectified linear unit (ReLU)**

$$\text{ReLU}(\mathbf{x}_i) = \max(\mathbf{x}_i, 0)$$

- Most commonly used

- **Sigmoid**

$$\sigma(\mathbf{x}_i) = \frac{1}{1 + e^{-\mathbf{x}_i}}$$

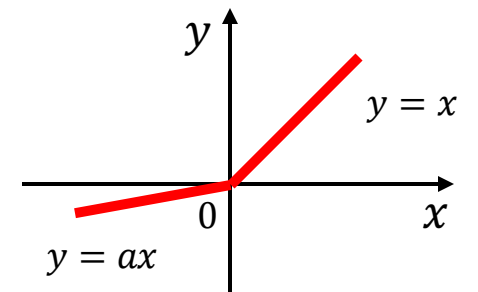
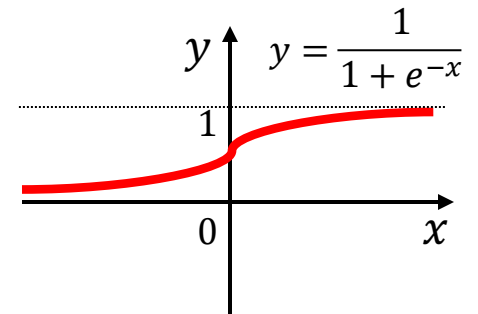
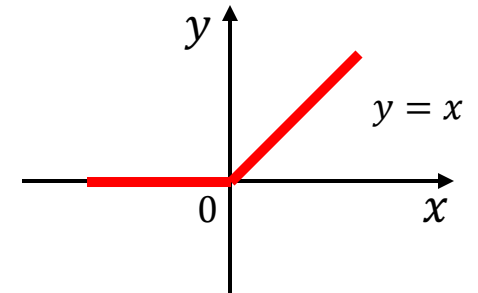
- Used only when you want to restrict the range of your embeddings

- **Parametric ReLU**

$$\text{PReLU}(\mathbf{x}_i) = \max(\mathbf{x}_i, 0) + a_i \min(\mathbf{x}_i, 0)$$

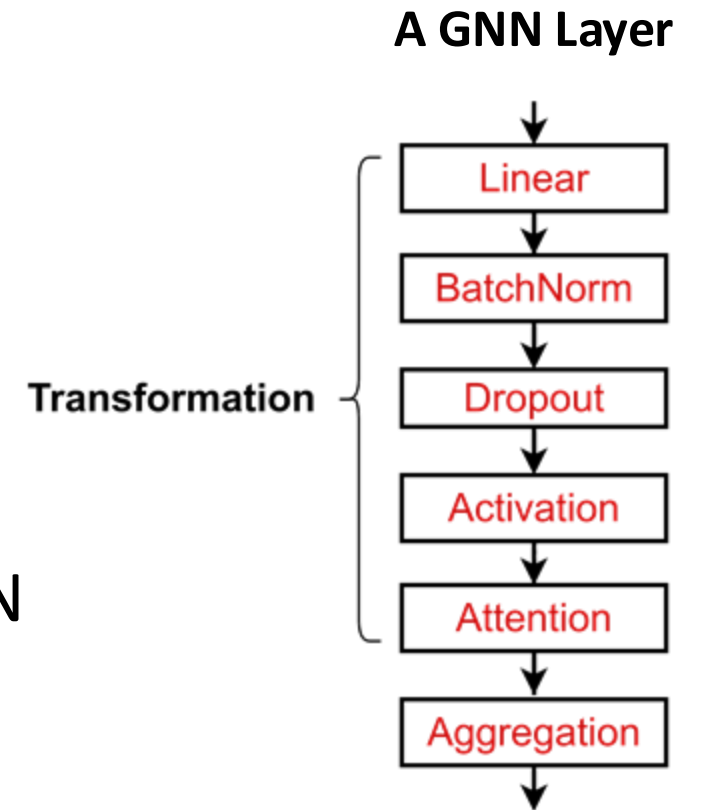
a_i is a trainable parameter

- Empirically performs better than ReLU



GNN Layer in Practice

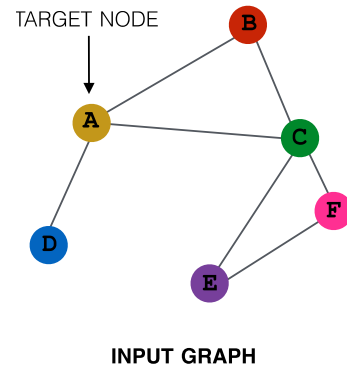
- **Summary:** Modern deep learning modules can be included into a GNN layer for better performance
- **Designing novel GNN layers is still an active research frontier!**
- **Suggested resources:** You can explore diverse GNN designs or try out your own ideas in [GraphGym](#)



Graph Neural Networks: Model II

Stacking Layers of a GNN

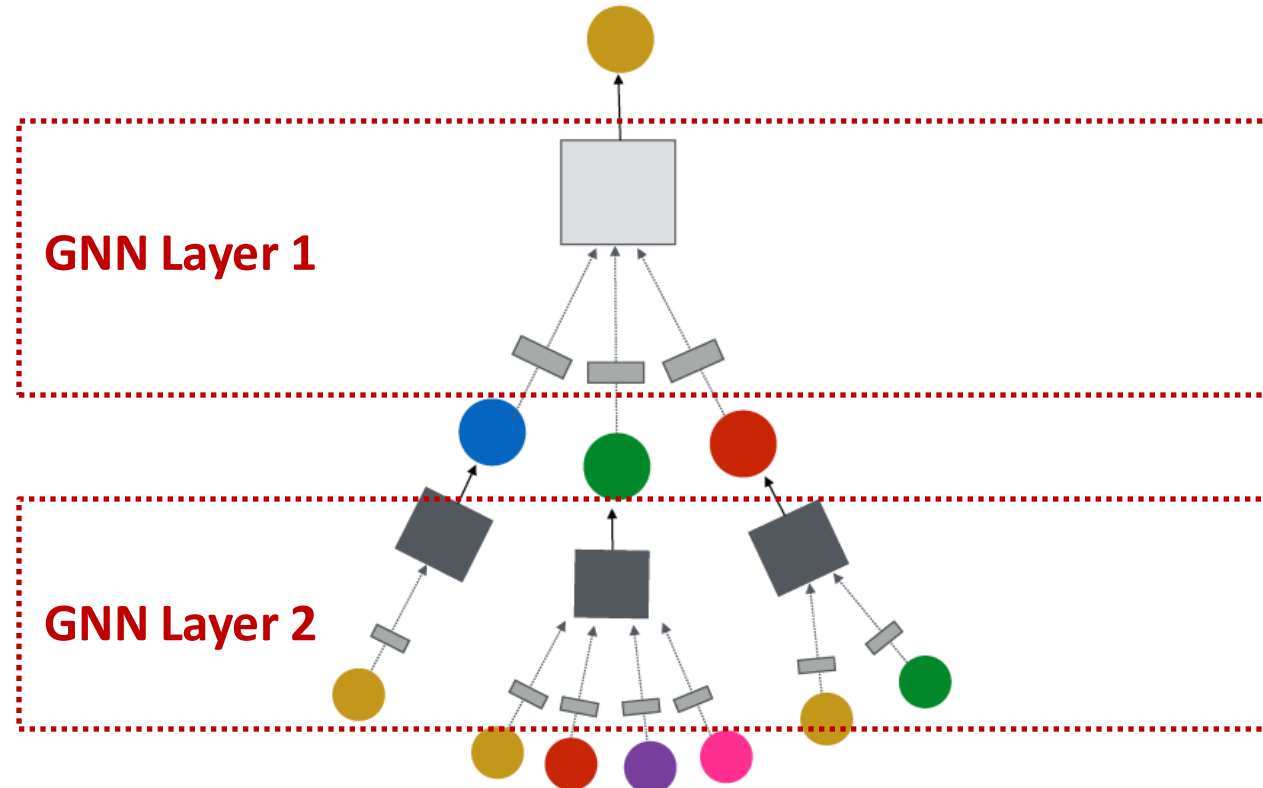
Stacking GNN Layers



How to connect GNN layers into a GNN?

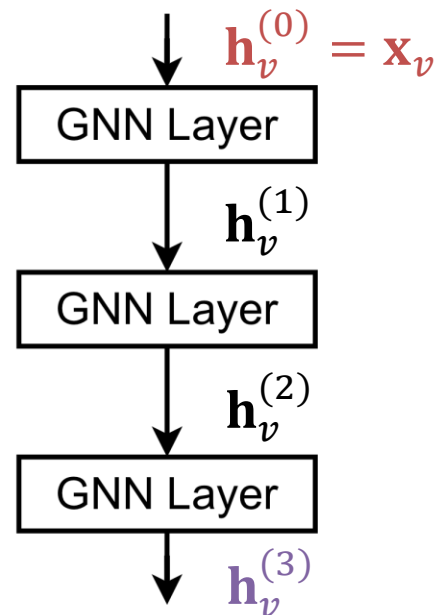
- Stack layers sequentially
- Ways of adding skip connections

(3) Layer connectivity



Stacking GNN Layers

- **How to construct a Graph Neural Network?**
 - **The standard way:** Stack GNN layers sequentially
 - **Input:** Initial raw node feature $\mathbf{x}_v$
 - **Output:** Node embeddings $\mathbf{h}_v^{(L)}$ after L GNN layers

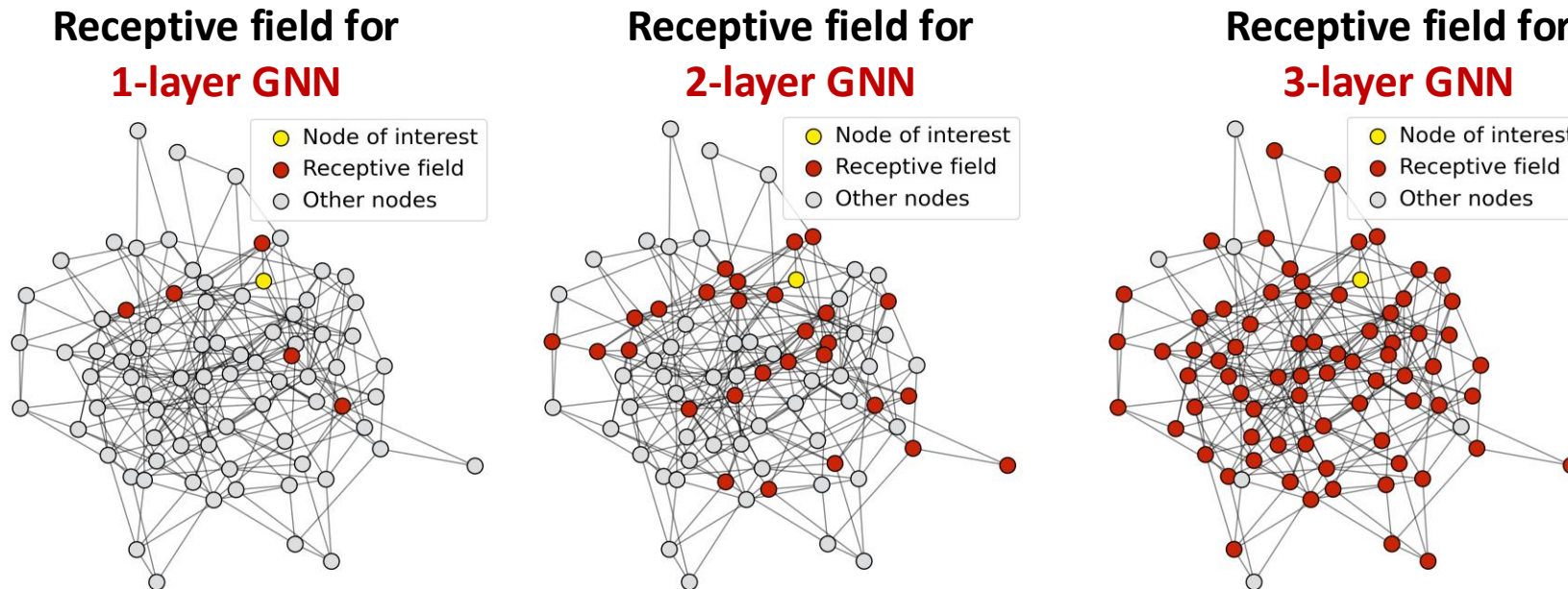


The Over-smoothing Problem

- The Issue of stacking many GNN layers
 - GNN suffers from **the over-smoothing problem**
- **The over-smoothing problem:** all the node embeddings converge to the same value
 - This is bad because we **want to use node embeddings to differentiate nodes**
- **Why does the over-smoothing problem happen?**

Receptive Field of a GNN

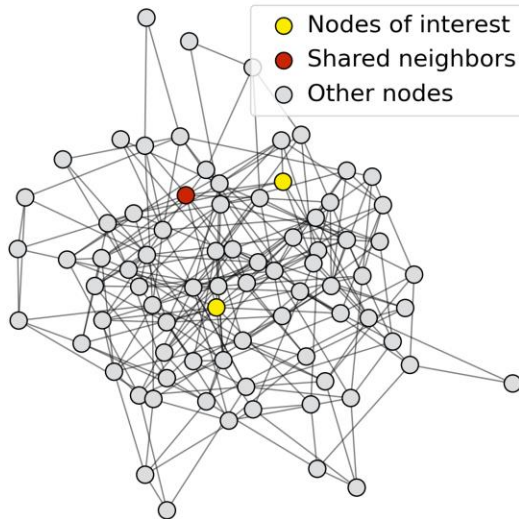
- **Receptive field:** the set of nodes that determine the embedding of a node of interest
 - In a K -layer GNN, each node has a receptive field of K -hop neighborhood



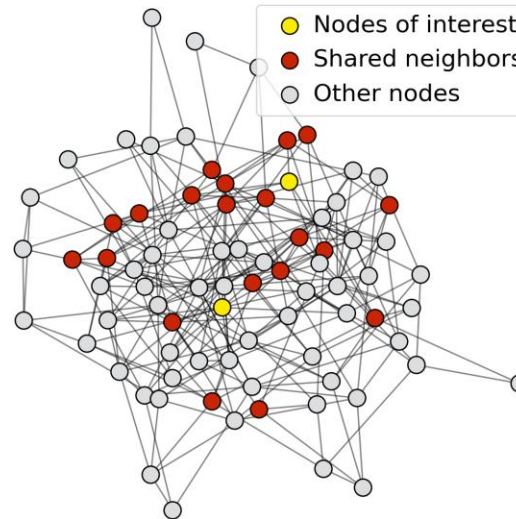
Receptive Field of a GNN

- **Receptive field overlap** for two nodes
 - **The shared neighbors quickly grows** when we increase the number of hops (num of GNN layers)

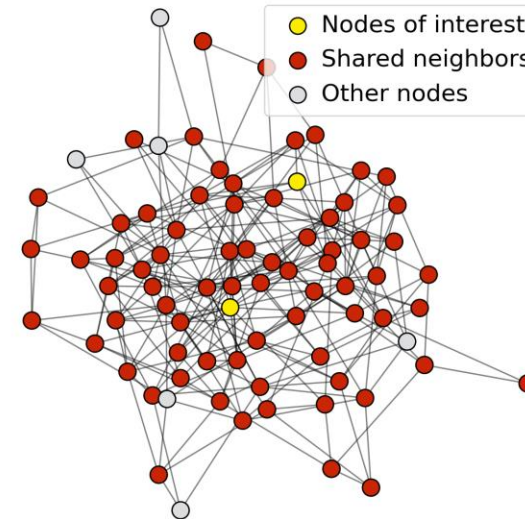
1-hop neighbor overlap
Only 1 node



2-hop neighbor overlap
About 20 nodes



3-hop neighbor overlap
Almost all the nodes!



Receptive Field & Over-smoothing

- We can explain over-smoothing via the notion of receptive field
 - We knew the embedding of a node is determined by its receptive field
 - If two nodes have highly-overlapped receptive fields, then their embeddings are highly similar
 - Stack many GNN layers → nodes will have highly-overlapped receptive fields → node embeddings will be highly similar → suffer from the over-smoothing problem
- Next: how do we overcome over-smoothing problem?

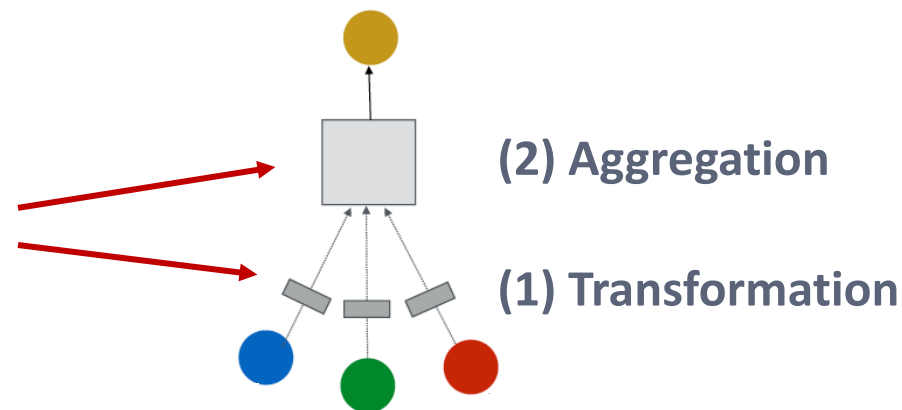
Design GNN Layer Connectivity

- What do we learn from the over-smoothing problem?
- Lesson 1: Be cautious when adding GNN layers
 - Unlike neural networks in other domains (CNN for image classification), **adding more GNN layers do not always help**
 - **Step 1: Analyze the necessary receptive field** to solve your problem. E.g., by computing the diameter of the graph
 - **Step 2:** Set number of GNN layers L to be a bit more than the receptive field we like. **Do not set L to be unnecessarily large!**
- Question: **How to make a shallow GNN more expressive?**

Expressive Power for Shallow GNNs

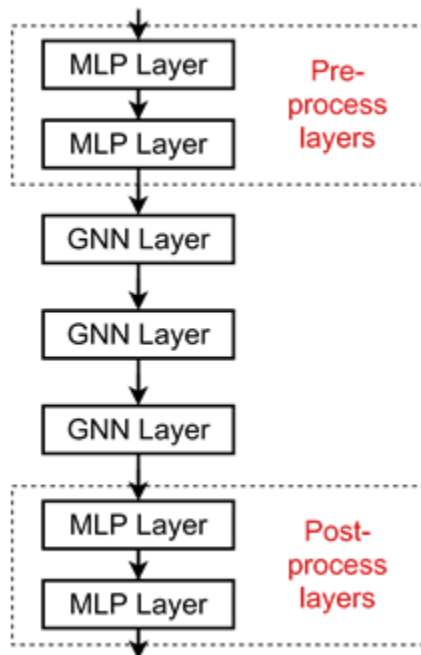
- **How to make a shallow GNN more expressive?**
- **Solution 1:** Increase the expressive power **within each GNN layer**
 - In our previous examples, each transformation or aggregation function only include one linear layer
 - We can **make aggregation / transformation become a deep neural network!**

If needed, each box could include a **3-layer MLP**



Expressive Power for Shallow GNNs

- **How to make a shallow GNN more expressive?**
- **Solution 2:** Add layers that do not pass messages
 - A GNN does not necessarily only contain GNN layers
 - E.g., we can add **MLP layers** (applied to each node) before and after GNN layers, as **pre-process layers** and **post-process layers**



Pre-processing layers: Important when encoding node features is necessary.

E.g., when nodes represent images/text

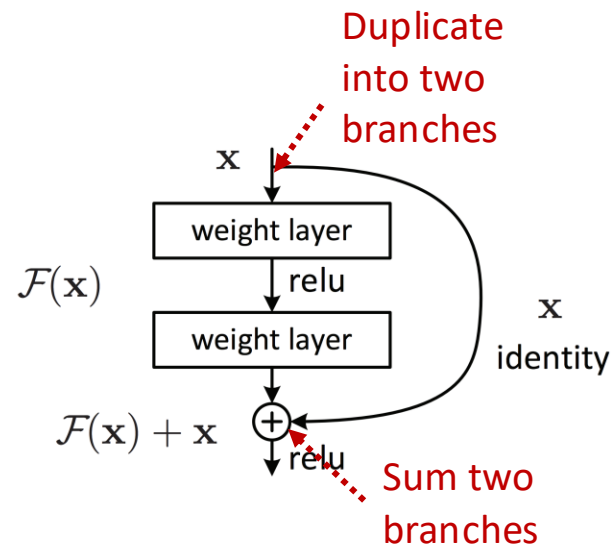
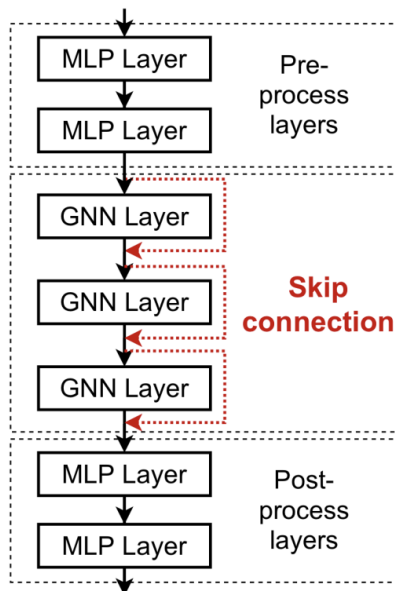
Post-processing layers: Important when reasoning / transformation over node embeddings are needed

E.g., graph classification, knowledge graphs

In practice, adding these layers works great!

Design GNN Layer Connectivity

- What if my problem still requires many GNN layers?
- Lesson 2: Add skip connections in GNNs
 - **Observation from over-smoothing:** Node embeddings in earlier GNN layers can sometimes better differentiate nodes
 - **Solution:** We can increase the impact of earlier layers on the final node embeddings, **by adding shortcuts in GNN**



Idea of skip connections:

Before adding shortcuts:

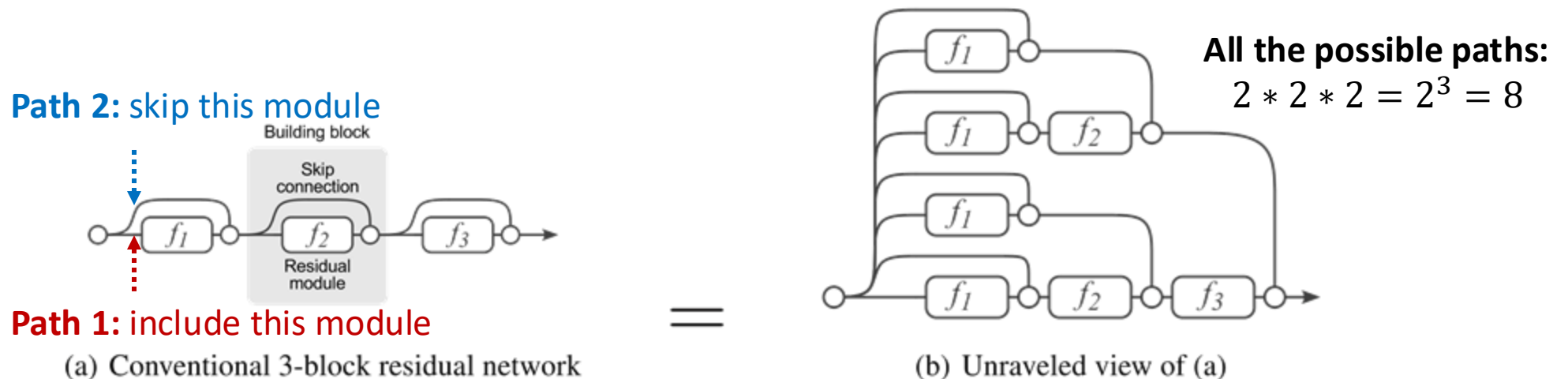
$$F(x)$$

After adding shortcuts:

$$F(x) + x$$

Idea of Skip Connections

- **Why do skip connections work?**
 - **Intuition:** Skip connections create **a mixture of models**
 - N skip connections $\rightarrow 2^N$ possible paths
 - Each path could have up to N modules
- We automatically get **a mixture of shallow GNNs and deep GNNs**



Example: GCN with Skip Connections

- A standard GCN layer

- $$\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in N(v)} \mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|} + \mathbf{B}^{(l)} \mathbf{h}_v^{(l-1)} \right)$$

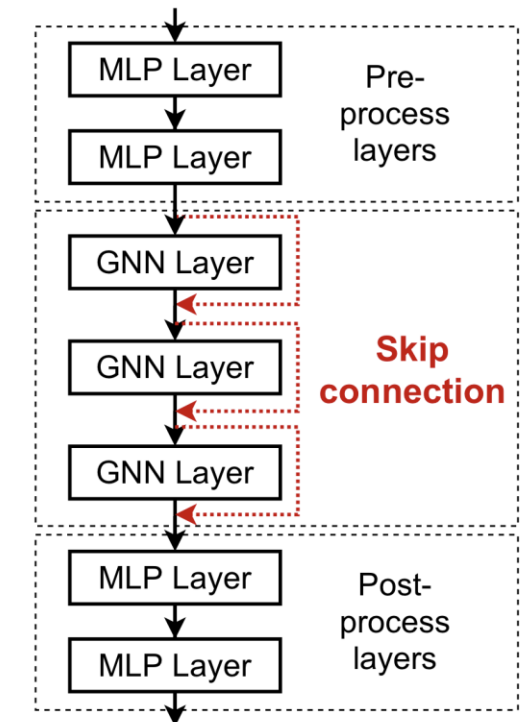
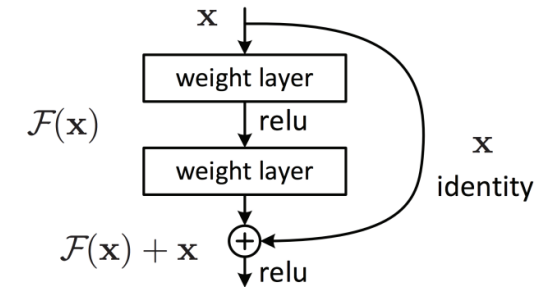
This is our $F(\mathbf{x})$

- A GCN layer with skip connection across layers

- Within layer skip connection + Across layer skip connection

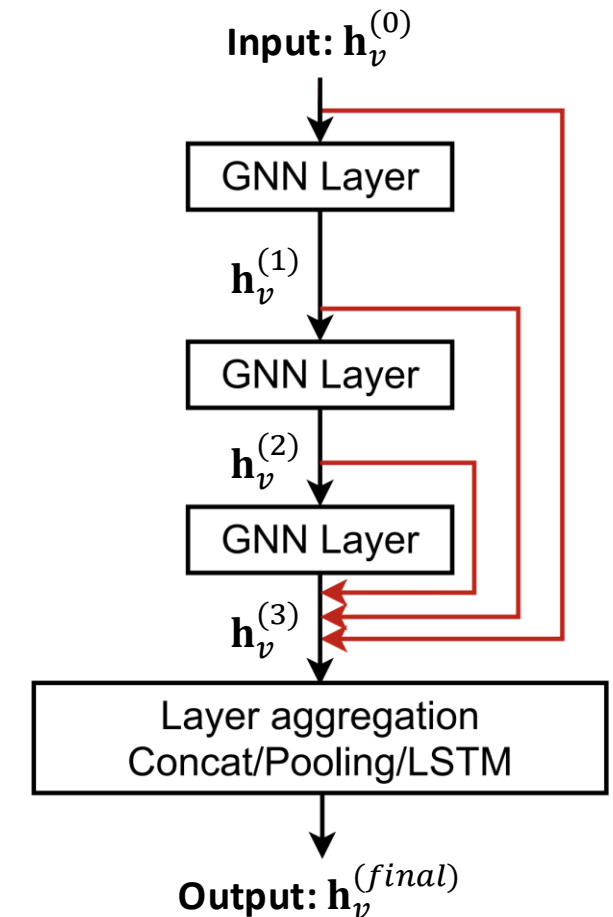
- $$\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in N(v)} \mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|} + \mathbf{B}^{(l)} \mathbf{h}_v^{(l-1)} \right) + \mathbf{h}_v^{(l-1)}$$

$F(\mathbf{x}) \quad + \quad \mathbf{x}$



Other Options of Skip Connections

- **Other options:** Directly skip to the last layer
 - The final layer directly **aggregates from the all the node embeddings** in the previous layers



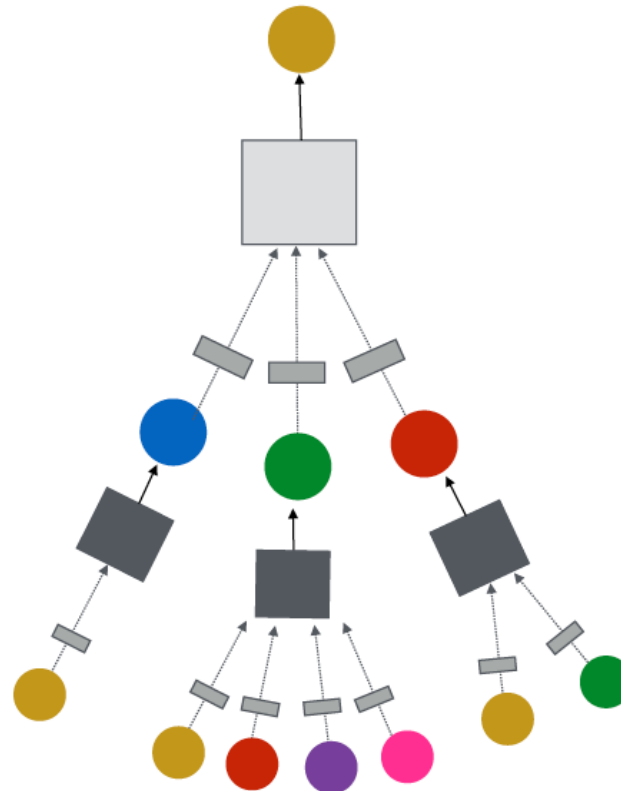
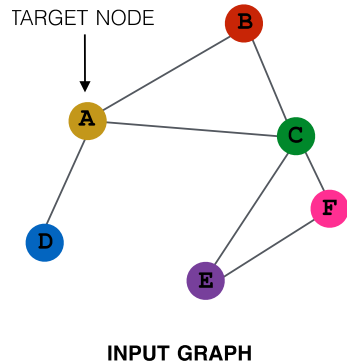
Graph Neural Networks: Model II

GNN Augmentation and Training

General GNN Framework

Idea: Raw input graph $\neq$ computational graph

- Graph feature augmentation
- Graph structure manipulation



(4) Graph manipulation

Why Manipulate Graphs

Our assumption so far has been

- **Raw input graph = computational graph**

Reasons for breaking this assumption

- **Feature level:**

- The input graph **lacks features** → feature augmentation

- **Structure level:**

- The graph is **too sparse** → inefficient message passing
 - The graph is **too dense** → message passing is too costly
 - The graph is **too large** → cannot fit the computational graph into a GPU
- It's just **unlikely that the input graph happens to be the optimal computation graph** for embeddings

Graph Manipulation Approaches

- **Graph Feature manipulation**

- The input graph **lacks features** → **feature augmentation**

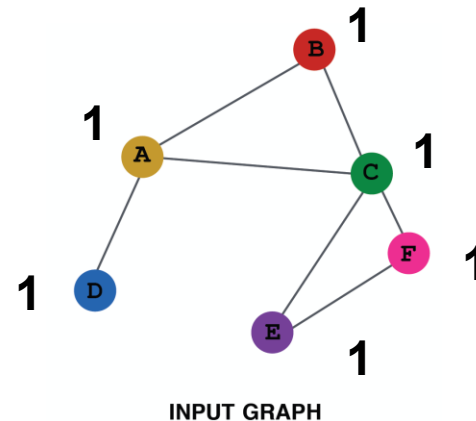
- **Graph Structure manipulation**

- The graph is **too sparse** → **Add virtual nodes / edges**
- The graph is **too dense** → **Sample neighbors when doing message passing**
- The graph is **too large** → **Sample subgraphs to compute embeddings**
 - Will cover later in lecture: Scaling up GNNs

Feature Augmentation on Graphs

Why do we need feature augmentation?

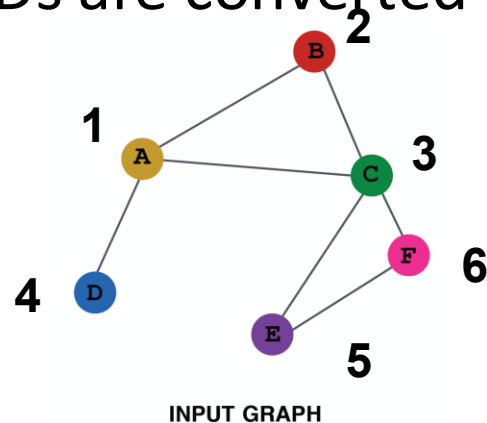
- **(1) Input graph does not have node features**
 - This is common when we only have the adj. matrix
- **Standard approaches:**
- **a) Assign constant values to nodes**



Feature Augmentation on Graphs

Why do we need feature augmentation?

- **(1) Input graph does not have node features**
 - This is common when we only have the adj. matrix
- **Standard approaches:**
- **b) Assign unique IDs to nodes**
 - These IDs are converted into **one-hot vectors**

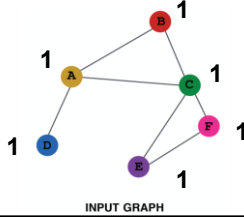
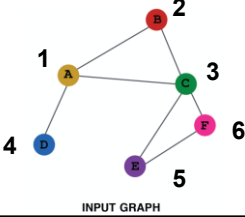


One-hot vector for node with ID=5

ID = 5
↓
[0, 0, 0, 0, 1, 0]
└──────────┘
Total number of IDs = 6

Feature Augmentation on Graphs

■ Feature augmentation: **constant** vs. **one-hot**

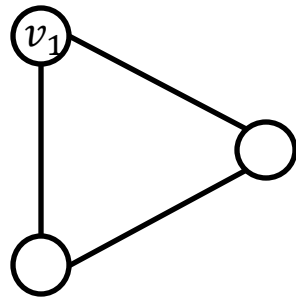
	Constant node feature  INPUT GRAPH	One-hot node feature  INPUT GRAPH
Expressive power	Medium. All the nodes are identical, but GNN can still learn from the graph structure	High. Each node has a unique ID, so node-specific information can be stored
Inductive learning (Generalize to unseen nodes)	High. Simple to generalize to new nodes: we assign constant feature to them, then apply our GNN	Low. Cannot generalize to new nodes: new nodes introduce new IDs, GNN doesn't know how to embed unseen IDs
Computational cost	Low. Only 1 dimensional feature	High. High dimensional feature, cannot apply to large graphs
Use cases	Any graph, inductive settings (generalize to new nodes)	Small graph, transductive settings (no new nodes)

Feature Augmentation on Graphs

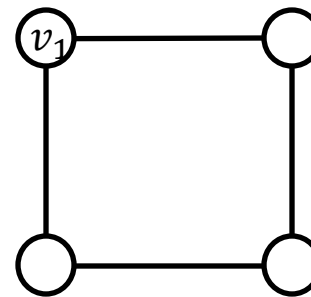
Why do we need feature augmentation?

- (2) Certain structures are hard to learn by GNN
- **Example:** Cycle count feature
 - Can GNN learn the length of a cycle that v_1 resides in?
 - **Unfortunately, no**

v_1 resides in a cycle with length 3



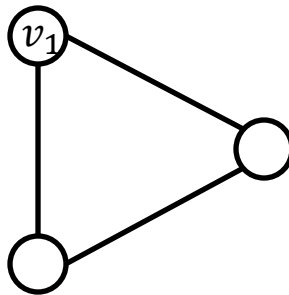
v_1 resides in a cycle with length 4



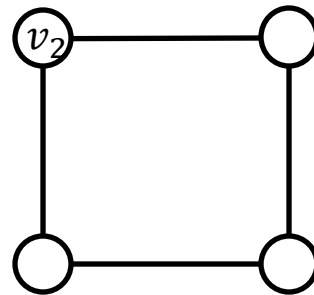
Feature Augmentation on Graphs

- v_1 cannot differentiate which graph it resides in
 - Because all the nodes in the graph have degree of 2
 - The computational graphs will be the same binary tree

v_1 resides in a cycle with length 3



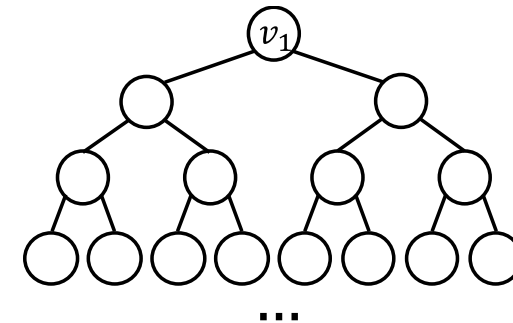
v_1 resides in a cycle with length 4



v_1 resides in a cycle with infinite length



The computational graphs for node v_1 are always the same



Feature Augmentation on Graphs

Why do we need feature augmentation?

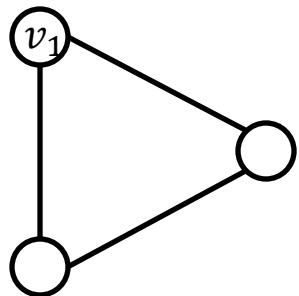
- (2) Certain structures are hard to learn by GNN
- **Solution:**
 - We can use **cycle count** as augmented node features

We start
from cycle
with length 0

Augmented node feature for v_1
[0, 0, 0, 1, 0, 0]



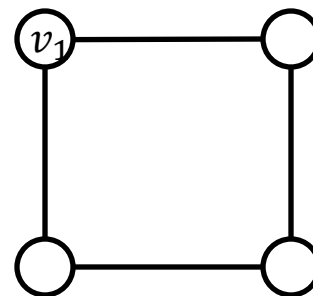
v_1 resides in a cycle with length 3



Augmented node feature for v_1
[0, 0, 0, 0, 1, 0]



v_1 resides in a cycle with length 4



Feature Augmentation on Graphs

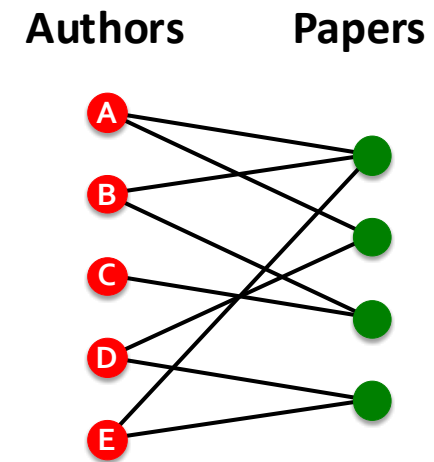
Why do we need feature augmentation?

- **(2) Certain structures are hard to learn by GNN**
- Other commonly used augmented features:
 - Degree distribution
 - Clustering coefficient
 - PageRank
 - Centrality
 - ...
- Any extracted features on graph can be used!

Add Virtual Nodes / Edges

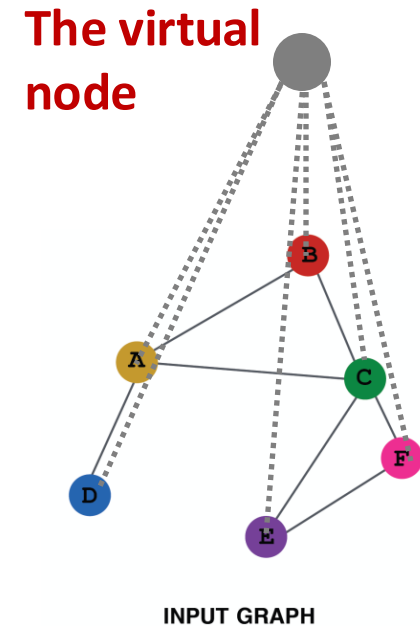
- **Motivation:** Augment sparse graphs
- **(1) Add virtual edges**
 - **Common approach:** Connect 2-hop neighbors via virtual edges
 - **Intuition:** Instead of using adj. matrix A for GNN computation, use $A + A^2$

- **Use cases:** Bipartite graphs
 - Author-to-papers (they authored)
 - 2-hop virtual edges make an author-author collaboration graph



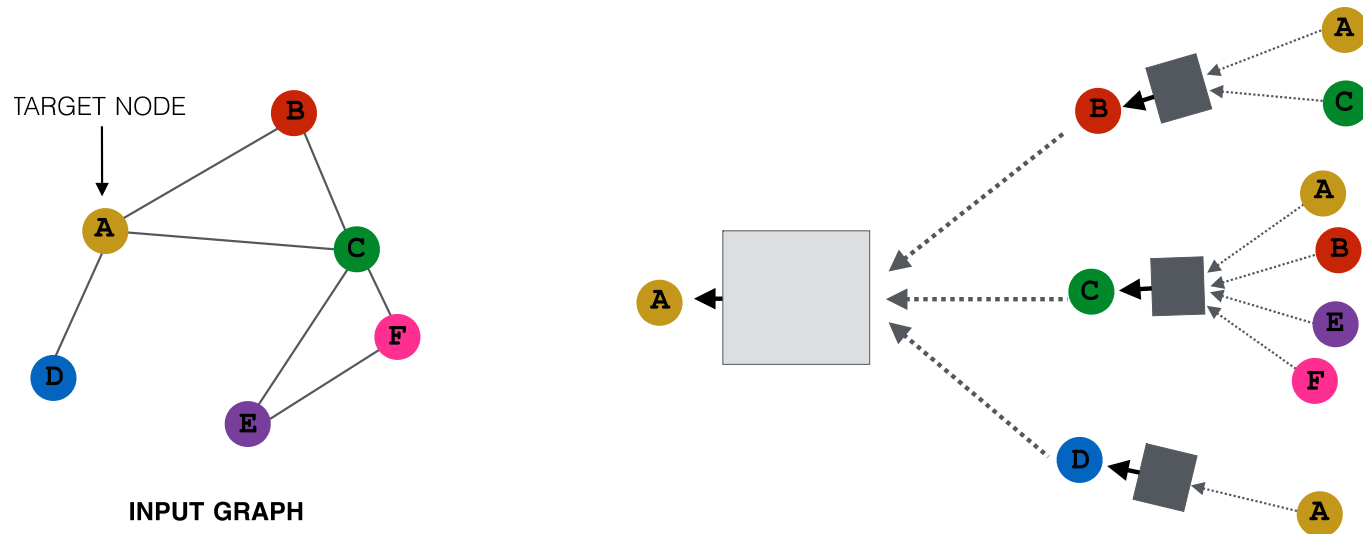
Add Virtual Nodes / Edges

- **Motivation:** Augment sparse graphs
- **(2) Add virtual nodes**
 - The virtual node will connect to all the nodes in the graph
 - Suppose in a sparse graph, two nodes have shortest path distance of 10
 - After adding the virtual node, **all the nodes will have a distance of 2**
 - Node A – Virtual node – Node B
 - **Benefits:** Greatly **improves message passing in sparse graphs**



Node Neighborhood Sampling

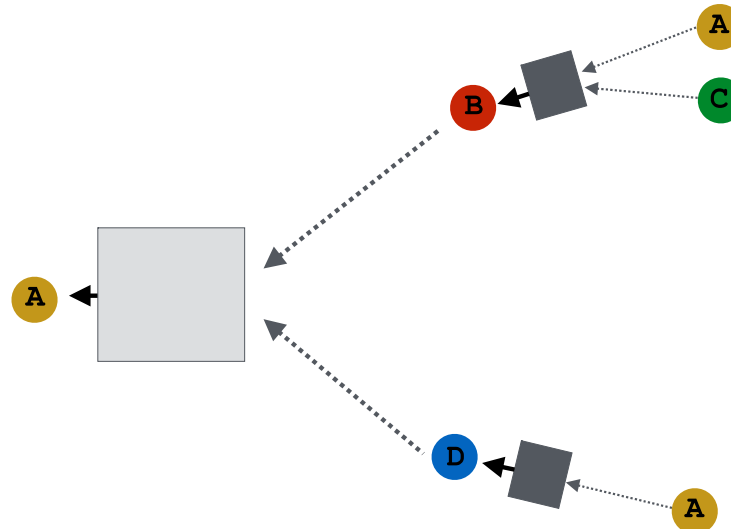
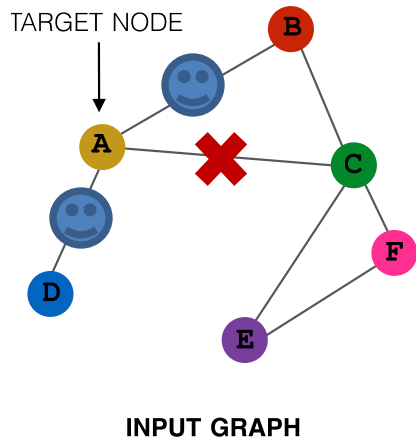
- **Previously:**
 - All the nodes are used for message passing



- **New idea:** (Randomly) sample a node's neighborhood for message passing

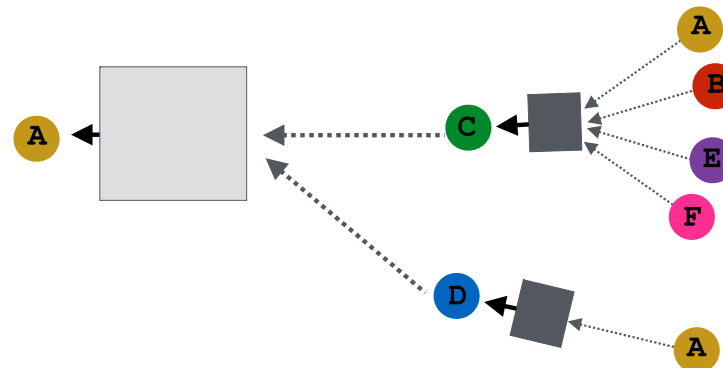
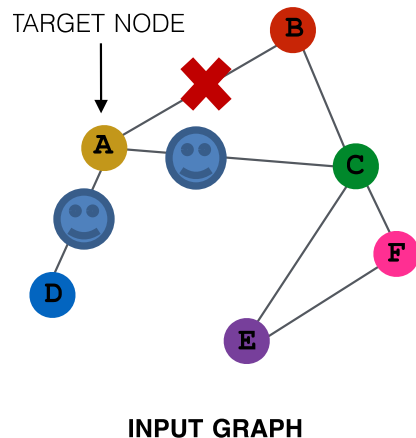
Neighborhood Sampling Example

- For example, we can randomly choose 2 neighbors to pass messages
 - Only nodes *B* and *D* will pass message to *A*



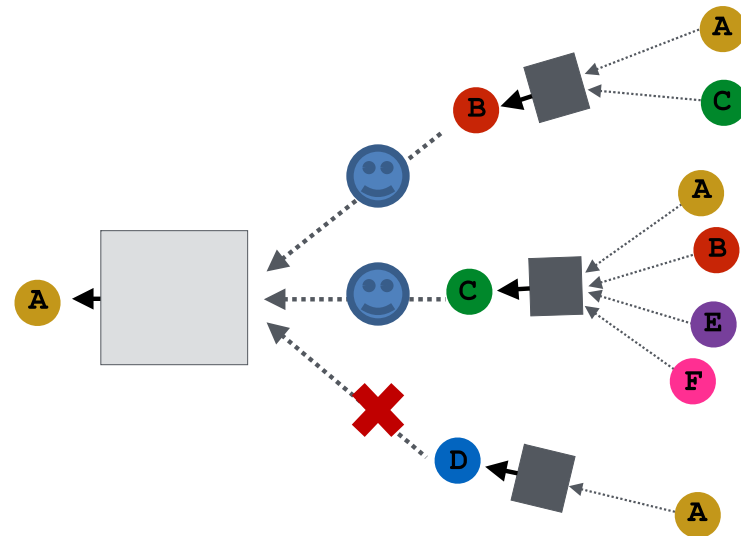
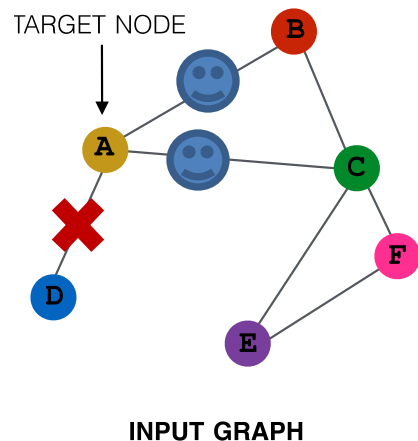
Neighborhood Sampling Example

- Next time when we compute the embeddings, we can sample different neighbors
 - Only nodes *C* and *D* will pass message to *A*



Neighborhood Sampling Example

- In expectation, we can get embeddings similar to the case where all the neighbors are used
 - **Benefits:** Greatly reduce computational cost
 - And in practice it works great!



Summary of the Lecture

- We introduce **a general GNN framework**:
 - **GNN Layer**:
 - Transformation + Aggregation
 - Classic GNN layers: GCN, GraphSAGE, GAT
 - **Layer connectivity**:
 - The over-smoothing problem
 - Solution: skip connections
 - **Graph Augmentation**:
 - Feature augmentation
 - Structure augmentation

Paper Discussion & Teammate Finding – Next lecture

Discussion Format & Objectives

Discussion Format & Objectives

- Our discussions will center on **sharing your Writing task 1**
 - **We will take attendance for this lecture (deduct 1% if you do not show up)**
- **5 research questions:**
 - What is the problem?
 - Why is it interesting and important?
 - Why is it hard?
 - Why hasn't it been solved before?
 - What are the key components of my approach and results?
- **1 Comparison table**
 - How these papers differ in different dimensions?

Schedule & Process

- **Next Wednesday Schedule**

- 3 sessions, ~20 minutes each
 - 6 people per group, seating suggestions will be provided
 - Enter your 3 paper choices on Google Sheets [[link](#)] **before the lecture**
- Briefly Introduce your research backgrounds and interests
- Present your paper choices and analysis and comment on others

- **Nominate interesting papers after the lecture**

- Fill in the details of recommended papers on Google Form (will be shared next Wednesday).

- **Find teammates for course project**

- **3 people per group** by default
- Fill in the names of team members on Google Sheets [[link](#)]